Session 10 Bayes Theorem(Basics)

Conditional Probability Update, Naive Bayes Principles & Real-World Applications

1. Fundamental Bayes Posterior Probability Function

Formula: `P(A|B) = [P(B|A) * P(A)] / P(B)` where P(A) is Prior, P(B|A) is Likelihood, and P(B) is Total Evidence.

```
def bayes_posterior(prior, likelihood, evidence): if evidence == 0: return 0.0 return (likelihood * prior) / evidence # Validation test case p_A = 0.05 # Prior P(A) p_B_given_A = 0.90 # Likelihood P(B|A) p_B = 0.12 # Marginal Evidence P(B) posterior = bayes_posterior(p_A, p_B_given_A, p_B) print(f"Calculated Posterior P(A|B): {posterior:.4f} ({posterior*100:.2f}%)")
```

FUNCTION OUTPUT

Calculated Posterior P(A|B): **0.3750 (37.50%)**

2. Swiggy Restaurant Categorization (Premium given Discount)

Scenario: 10% premium restaurants (80% offer discounts); 90% regular restaurants (20% offer discounts). Find `P(Premium | Discount)`.

```
# 1. Prior Probabilities p_premium = 0.10 # P(Premium) p_regular = 0.90 # P(Regular) # 2. Likelihoods p_discount_given_premium = 0.80 # P(Discount | Premium) p_discount_given_regular = 0.20 # P(Discount | Regular) # 3. Total Marginal Evidence: P(Discount) p_discount = (p_discount_given_premium * p_premium) + (p_discount_given_regular * p_regular) # P(Discount) = (0.80 * 0.10) + (0.20 * 0.90) = 0.08 + 0.18 = 0.26 # 4. Posterior Calculation p_premium_given_discount = (p_discount_given_premium * p_premium) / p_discount print(f"Total Evidence P(Discount): {p_discount:.2f}") print(f"Posterior P(Premium | Discount): {p_premium_given_discount:.4f}")
```

PROBABILITY BREAKDOWN

- **P(Discount) = 26.0%** of all restaurants offer discounts.
- **P(Premium | Discount) = 0.08 / 0.26 = 30.77%**.
- **Conclusion:** Seeing a discount increases the likelihood that a restaurant is premium from a prior of 10% up to a posterior of 30.77%.

3. Email Spam Filter Simulation (Keyword: 'Free')

Dataset & Parameters: 20 emails (5 Spam, 15 Non-Spam). P('Free'|Spam) = 0.80, P('Free'|Non-Spam) = 0.10.

```
emails = [ ("Get free coins now", "spam"), ("Claim free trial", "spam"), ("Win a free car", "spam"), ("Urgent free gift", "spam"), ("Free access limited", "spam"), ("Meeting tomorrow", "ham"), ("Project report attached", "ham"), ("Are you free for coffee?", "ham"), ("Invoice updates", "ham"), ("Weekly newsletter", "ham"), ("Team lunch call", "ham"), ("Code review request", "ham"), ("Flight confirmation", "ham"), ("Doctor appointment", "ham"), ("Subscription renewed", "ham"), ("Weekend plans", "ham"), ("Design feedback", "ham"), ("Python course notes", "ham"), ("Grocery list", "ham"), ("Gym membership", "ham") ] # Empirical Prior Probabilities p_spam = 5 / 20 # 0.25 p_ham = 15 / 20 # 0.75 # Likelihoods p_free_given_spam = 0.80 p_free_given_ham = 0.10 # Total Evidence P('Free') p_free = (p_free_given_spam * p_spam) + (p_free_given_ham * p_ham) # 0.20 + 0.075 = 0.275 p_spam_given_free = (p_free_given_spam * p_spam) / p_free print(f"P(Spam | 'free'): {p_spam_given_free:.4f} ({p_spam_given_free*100:.2f}%)")
```

SPAM FILTER RESULT

P(Spam | 'free'): **0.7273 (72.73%)**

4. Real-World AI Example: Flipkart Return Fraud Detection

LLM Prompt Used: *"Generate a real-world Bayes theorem scenario for Flipkart calculating the probability that an item return request is fraudulent given that the user opened an account within the last 24 hours."*

```
# Flipkart Scenario Setup: # Prior P(Fraud) = 2% of total returns # P(NewAccount | Fraud) = 60% of fraudulent returns come from accounts <24h old # P(NewAccount | Genuine) = 5% of genuine returns come from accounts <24h old p_fraud = 0.02 p_genuine = 0.98 p_new_given_fraud = 0.60 p_new_given_genuine = 0.05 # Evidence P(NewAccount) p_new_account = (p_new_given_fraud * p_fraud) + (p_new_given_genuine * p_genuine) # 0.012 + 0.049 = 0.061 # Bayes Posterior p_fraud_given_new = (p_new_given_fraud * p_fraud) / p_new_account print(f"P(Fraud | New Account): {p_fraud_given_new:.4f} ({p_fraud_given_new*100:.2f}%)")
```

E-COMMERCE SECURITY TAKEAWAY

While the baseline return fraud rate is only 2%, if a return is requested from an account created less than 24 hours ago, the posterior probability jumps to **19.67%**, flagging the transaction for automated manual verification.